

Java Data Types

The blueprint for variable storage

1. The Two Main Families

Primitive Types

Simple values stored directly in memory. Fast and efficient.

byte, short, int, long, float, double, boolean, char

Non-Primitive Types

Complex objects that store references (addresses) to data.

String, Arrays, Classes, Interfaces

2. Integer & Decimal Primitives

Type	Size	Best Use Case	Example
byte	1 Byte	Save memory in large arrays	127
int	4 Bytes	Standard for whole numbers	100000
long	8 Bytes	Vast numbers (billions+)	15000L
float	4 Bytes	Simple decimals	19.99f
double	8 Bytes	High precision decimals	3.14159

 Suffix Requirement:

- Use **L** for long values (e.g., 5000000000L)
- Use **f** for float values (e.g., 10.5f)

Codehive

3. Logic & Character Types

Boolean (Conditional)

```
boolean isFun = true;
```

Used for binary "yes/no" or "on/off" logic.

Char (Single Symbols)

```
char grade = 'A';
```

Must use **single quotes**. Represents one UTF-16 character.

4. Comparison Recap

Feature	Primitive types	Non-Primitive types
Storage	Stores values directly	Stores reference (memory address)
Methods	No methods available	Can call methods to perform actions
Nullability	Always has a value	Can be null
Starting Case	Lowercase (int)	Uppercase (String)

5. Implementation Example

```
String name = "Liam";  
int age = 20;  
double height = 5.9;  
char grade = 'A';  
boolean isStudent = true;  
  
System.out.println("Record created for " + name);
```



Key Rule: Type Strictness

Once you declare a type, you cannot change it.

```
int x = 10; x = "Hi"; ← Error!
```

Choosing the right type saves memory and prevents "type mismatch" bugs.

Codehive